

SymPy Bug Report

1 Bug 8: K-Weighted Alternating Binomial Sum Misses the $n = 1$ Case

1.1 Minimal Reproducer

```
from sympy import binomial, summation, symbols

n = symbols("n", integer=True, nonnegative=True)
k = symbols("k", integer=True)

closed = summation((-1)**k*k*binomial(n, k), (k, 0, n))
direct_n1 = sum((-1)**j*j*binomial(1, j) for j in range(0, 2))

print("closed form =", closed)
print("closed at n=1 =", closed.subs(n, 1))
print("direct finite sum at n=1 =", direct_n1)
print("difference =", closed.subs(n, 1) - direct_n1)
```

1.2 Observed Behavior

```
closed form = 0
closed at n=1 = 0
direct finite sum at n=1 = -1
difference = 1
```

SymPy returns the unconditional closed form 0. Substituting $n = 1$ into that result gives 0, while direct evaluation of the finite sum at $n = 1$ gives -1 . The returned scalar therefore loses a parameter value allowed by the declared assumption $n \in \mathbb{Z}_{\geq 0}$.

1.3 Expected Behavior

For a nonnegative integer symbol n , the result should preserve the exceptional $n = 1$ value. A correct result for the representative sum is

$$\text{Piecewise}((-1, \text{Eq}(n, 1)), (0, \text{Ne}(n, 1))), \quad n \in \mathbb{Z}_{\geq 0},$$

or any equivalent expression that evaluates to -1 at $n = 1$ and 0 at all other nonnegative integer values of n .

1.4 Mathematical Explanation

Differentiate the binomial identity

$$(1+t)^n = \sum_{k=0}^n \binom{n}{k} t^k$$

with respect to t . This gives

$$n(1+t)^{n-1} = \sum_{k=0}^n k \binom{n}{k} t^{k-1}.$$

Multiplying by t and setting $t = -1$ yields

$$\sum_{k=0}^n (-1)^k k \binom{n}{k} = -n(1-1)^{n-1}.$$

For $n > 1$, this is 0. For $n = 1$, however, the differentiated identity must be evaluated as the original finite sum:

$$\sum_{k=0}^1 (-1)^k k \binom{1}{k} = 0 \binom{1}{0} - 1 \binom{1}{1} = -1.$$

The case $n = 0$ is also 0, since the only summand contains the factor $k = 0$. Thus the scalar 0 is not an equivalent simplification over the declared nonnegative-integer domain; it has the wrong value at $n = 1$.

1.5 Source-Level Diagnosis

The diagnosis locates the failure in SymPy's concrete summation path. The public call constructs a `Sum` and calls `doit`. The relevant source files are:

```
sympy/concrete/summations.py  
sympy/concrete/gosper.py
```

Since the range length is symbolic, evaluation reaches `eval_sum` and then `eval_sum_symbolic`. The traced call path reaches the Gosper branch around `sympy/concrete/summations.py` lines 1236–1256, which calls `gosper_sum`.

For the summand $(-1)^k k \binom{n}{k}$, the traced Gosper certificate is

$$g = -\frac{k-1}{n-1}.$$

This certificate has a pole at the parameter value $n = 1$. The diagnosis identifies the immediate faulty endpoint calculation in `sympy/concrete/gosper.py`:

```
result = (f*(g + 1)).subs(k, b) - (f*g).subs(k, a)
```

`gosper_sum` substitutes the symbolic endpoints and simplifies the generic telescoping result to the scalar 0, without recording that the certificate is invalid at $n = 1$. The caller has logic to wrap some results in `Piecewise` when visible denominators remain in `Mul` or `Add` results, but here the endpoint expression has already simplified to 0. As a result, the excluded parameter value is no longer visible to `eval_sum_symbolic`, and the generic value becomes the final answer.

The suggested fix direction is to track parameter denominators introduced by `gosper_term` or `gosper_sum`, and either emit a conditional result or fall back to direct evaluation at excluded parameter values. For this example, the final result needs an $n = 1$ branch.

1.6 Additional Failing Instances

The independent verification checks the same endpoint failure for

$$\sum_{k=0}^n (-1)^k k^p \binom{n}{k}, \quad p = 1, 2, 3, 4, 5, 6.$$

For each listed power, SymPy returns a closed form whose value at $n = 1$ is 0. The direct finite sum at $n = 1$ is the same in all cases with $p \geq 1$, because the $k = 0$ term is $0^p \binom{1}{0} = 0$ and the $k = 1$ term is $(-1)1^p \binom{1}{1} = -1$.

Input / Expression	SymPy behavior	Expected behavior
$\sum_{k=0}^n (-1)^k k \binom{n}{k}$, evaluated at $n = 1$	Closed form gives 0	Direct sum gives -1
$\sum_{k=0}^n (-1)^k k^2 \binom{n}{k}$, evaluated at $n = 1$	Closed form gives 0	Direct sum gives -1
$\sum_{k=0}^n (-1)^k k^3 \binom{n}{k}$, evaluated at $n = 1$	Closed form gives 0	Direct sum gives -1
$\sum_{k=0}^n (-1)^k k^4 \binom{n}{k}$, evaluated at $n = 1$	Closed form gives 0	Direct sum gives -1
$\sum_{k=0}^n (-1)^k k^5 \binom{n}{k}$, evaluated at $n = 1$	Closed form gives 0	Direct sum gives -1
$\sum_{k=0}^n (-1)^k k^6 \binom{n}{k}$, evaluated at $n = 1$	Closed form gives 0	Direct sum gives -1

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. Besides general documentation and background for Gosper summation, the open issue [sympy/sympy#29731](#) (“A summation that evaluates to 0 even though it can be nonzero depending on the variable”, created 2026-05-02) is a near-exact public report of this same behavior: its reproducer returns the scalar 0 for an (-1) -weighted i^k binomial sum that is nonzero at the singular parameter value, with the same `gosper_sum` lost-endpoint-exception root cause. That issue predates this bundle, so the deduplication verdict is a new instance of an already-reported failure family: symbolic Gosper summation can require parameter conditions, and this candidate contributes the explicit $(-1)^k k \binom{n}{k}$ instantiation, the missing $n = 1$ branch, and the k^p generalization, all of which remain individually actionable as a cross-link to #29731 and as a regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import binomial, summation, symbols

n = symbols("n", integer=True, nonnegative=True)
k = symbols("k", integer=True)
CASES = [1, 2, 3, 4, 5, 6]

@pytest.mark.parametrize("power", CASES)
def test_weighted_alternating_binomial_sum_preserves_n1_exception(power):
    closed = summation((-1)**k*k**power*binomial(n, k), (k, 0, n))
    direct = sum((-1)**j*(j**power)*binomial(1, j) for j in range(2))
    assert closed.subs(n, 1) == direct
```